**AMRITA VISHWA VIDYAPEETHAM**
School of Artificial Intelligence & Data Engineering

# CAMPUSOS: SMART CAMPUS DIGITAL TWIN & UNIVERSITY MANAGEMENT SYSTEM

Course Code: 23AIE303 — Database Management Systems | Final Project Report



| | |
|---|---|
| **Author / Student Name:** | Sasank (CB.SC.U4AIE24026) |
| **Roll Number:** | CB.SC.U4AIE24026 |
| **Degree & Program:** | B.Tech in Artificial Intelligence & Data Engineering |
| **Course Instructor:** | Faculty of Computer Science & Engineering |
| **Live Cloud Deployment URL:** | https://campusos-sasank.azurewebsites.net |
| **GitHub Repository URL:** | https://github.com/Shashank01177/smart-campus-cloud-portal |
| **Cloud Platform & Database:** | Microsoft Azure App Service \| PostgreSQL 18 Flexible Server |
| **Submission Date:** | October 2026 |



## 1. Introduction and Problem Statement

**1.1 Executive Summary:** Modern collegiate ecosystems involve dozens of interconnected operational departments—including academic admissions, course enrollments, attendance accounting, examination evaluations, faculty schedules, physical hostel room allocations, library book loans, and corporate placement drives. In traditional university management setups, these domains operate in isolated silos, causing high data redundancy, concurrency anomalies, inconsistent student records, and lack of real-time visibility.

**1.2 Problem Statement:** Design, model, implement, and deploy a centralized, production-grade Relational Database Management System (RDBMS) paired with an interactive 3D digital twin interface that provides seamless Role-Based Access Control (RBAC) across Students, Faculty, and Administrators. The system must enforce comprehensive data integrity (Domain, Entity, Referential), eliminate update/deletion anomalies through 3NF/BCNF normalization, execute high-throughput complex SQL analytical queries, and be hosted natively on Microsoft Azure Cloud.

**1.3 Key Objectives:**

- **Relational Architecture:** Model 25 distinct relational entities with strict primary/foreign key hierarchies and cascading integrity constraints.
- **Normalization & Normal Forms:** Eliminate functional dependency violations to achieve Boyce-Codd Normal Form (BCNF) across all operational schemas.

- **Role-Based Security:** Implement administrative isolation where sensitive mutation operations (such as student enrollment additions and deletions) are strictly restricted to verified Admin roles, while Faculty and Students maintain appropriate read-only or scoped access.
- **Cloud Native Deployment:** Host PostgreSQL 18 Flexible Server and Python/Flask backend on Microsoft Azure, containerized via Docker and monitored in real-time through Azure Application Insights.

## 2. Conceptual and Logical Database Design

The conceptual design translates high-level campus organizational rules into an Entity-Relationship model comprising strong entities, weak entities, and associative entities representing many-to-many relationships. Logical database design maps these entities into normalized relational tables.

**Core Functional Domains Modeled:**

1. **Academic Core:** department (1:N) program (1:N) student; department (1:N) faculty; department (1:N) course.
2. **Enrollment & Instruction:** student_course_enrollment (M:N between Student and Course) and faculty_course (M:N between Faculty and Course).
3. **Academic Tracking:** attendance (Daily student tracking per course) and marks linked with examination schemas.
4. **Campus Facilities:** hostel (1:N) hostel_room (1:1 active) hostel_allocation; library_book (1:N) library_transaction.
5. **Career & Administration:** company (1:N) placement_drive (1:N) placement_registration; fee ledger; app_user RBAC authentication.

## 3. Normalization and Functional Dependencies

To guarantee data consistency, avoid data redundancy, and prevent Insertion, Deletion, and Modification anomalies, the relational schema was rigorously analyzed against Functional Dependencies (FDs) from 1NF through BCNF.

| Normal Form | Constraint / Condition Enforced | CampusOS Implementation Example |
|---|---|---|
| **1NF** (First Normal Form) | Atomic attribute values, unique column names, identification of a primary key. | Multi-valued attributes like course enrollments, student phone numbers, or qualifications were decomposed into distinct atomic columns or normalized associative tables (student_course_enrollment). |
| **2NF** (Second Normal Form) | Must be in 1NF + No partial functional dependencies (all non-prime attributes fully functionally dependent on whole candidate key). | In student_course_enrollment(student_id, course_id), attributes like course_name or credits were not stored; they reside solely in course referenced by course_id. |
| **3NF** (Third Normal Form) | Must be in 2NF + No transitive dependencies (non-prime attributes must not determine other non-prime attributes; $X \rightarrow Y$ implies X is superkey or Y is prime). | In student, department_name and hod_id are not stored directly. The table stores foreign key department_id, eliminating transitive dependency: student_id $\rightarrow$ department_id $\rightarrow$ department_name. |
| **BCNF** (Boyce-Codd NF) | For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. | In hostel_room, room numbers are scoped by hostel_id. Primary key is room_id, with candidate key (hostel_id, room_number). All determinants are candidate keys. |

# 4. Advanced SQL Implementation and Query Design

CampusOS demonstrates advanced SQL techniques utilizing multi-table joins, subqueries, grouping sets, conditional counting, null handling, and aggregated analytics across hundreds of thousands of production records.

## 4.1 Complex Analytical Query 1: Comprehensive Student Academic & Attendance Analytics

*Purpose:* Aggregates student demographic info, department, enrolled courses count, grade average, total lectures conducted, lectures attended, and calculated attendance percentage.

```
SELECT
    s.roll_number,
    s.first_name || ' ' || s.last_name AS student_name,
    d.department_name,
    s.cgpa,
    COUNT(DISTINCT sce.course_id) AS enrolled_courses,
    ROUND(AVG(m.marks_obtained), 2) AS avg_marks,
    COUNT(a.attendance_id) AS total_sessions,
    ROUND((COUNT(CASE WHEN a.status = 'Present' THEN 1 END)::numeric /
           NULLIF(COUNT(a.attendance_id), 0)) * 100, 2) AS attendance_pct
FROM student s
JOIN department d ON s.department_id = d.department_id
LEFT JOIN student_course_enrollment sce ON s.student_id = sce.student_id
LEFT JOIN marks m ON s.student_id = m.student_id
LEFT JOIN attendance a ON s.student_id = a.student_id
WHERE s.status = 'Active'
GROUP BY s.student_id, s.roll_number, s.first_name, s.last_name, d.department_name, s.cgpa
ORDER BY s.cgpa DESC LIMIT 10;
```

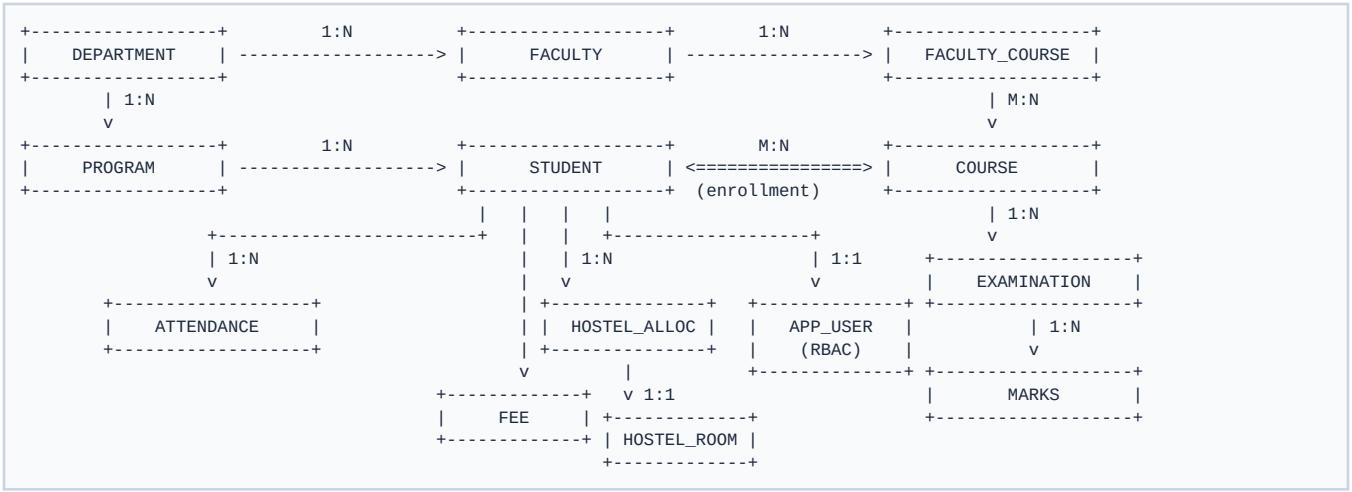
## 4.2 Complex Analytical Query 2: Hostel Bed Capacity & Real-Time Vacancy Ledger

*Purpose:* Calculates total block rooms, aggregate bed capacity, current occupied beds, and remaining vacancy per hostel block.

```
SELECT
    h.hostel_name,
    h.gender,
    COUNT(hr.room_id) AS total_rooms,
    SUM(hr.capacity) AS total_bed_capacity,
    COUNT(ha.allocation_id) AS occupied_beds,
    (SUM(hr.capacity) - COUNT(ha.allocation_id)) AS vacant_beds,
    ROUND((COUNT(ha.allocation_id)::numeric / NULLIF(SUM(hr.capacity), 0)) * 100, 2) AS occupancy_rate
FROM hostel h
JOIN hostel_room hr ON h.hostel_id = hr.hostel_id
LEFT JOIN hostel_allocation ha ON hr.room_id = ha.room_id AND ha.vacation_date IS NULL
GROUP BY h.hostel_id, h.hostel_name, h.gender
ORDER BY occupancy_rate DESC;
```

# 5. ER Diagram and UML Diagram

**5.1 Entity-Relationship (ER) Architecture:** The diagram illustrates cardinalities, participation constraints, and foreign key relationships across major campus entities.




## 6. Relational Schema Mapping

Below is the relational mapping of tables with primary keys (underlined) and foreign keys (*italicized with FK marker*):

- **DEPARTMENT** (department_id, department_name, department_code, *hod_id [FK]*, established_year, contact_email)
- **PROGRAM** (program_id, *department_id [FK]*, program_name, degree_level, duration_years, total_semesters)
- **FACULTY** (faculty_id, *department_id [FK]*, first_name, last_name, email, phone, designation, joining_date, salary)
- **STUDENT** (student_id, roll_number, first_name, last_name, email, phone, gender, dob, *department_id [FK]*, *program_id [FK]*, admission_year, current_year, current_semester, cgpa, status)
- **COURSE** (course_id, course_code, course_name, *department_id [FK]*, *program_id [FK]*, credits, semester_number, course_type)
- **FACULTY_COURSE** (allocation_id, *faculty_id [FK]*, *course_id [FK]*, academic_year, semester_type)
- **STUDENT_COURSE_ENROLLMENT** (enrollment_id, *student_id [FK]*, *course_id [FK]*, enrollment_date, status)
- **ATTENDANCE** (attendance_id, *student_id [FK]*, *course_id [FK]*, date, status, *marked_by [FK]*)
- **EXAMINATION** (exam_id, *course_id [FK]*, exam_type, exam_date, max_marks, weightage_percent)
- **MARKS** (mark_id, *student_id [FK]*, *exam_id [FK]*, marks_obtained, grade)
- **FEE** (fee_id, *student_id [FK]*, fee_type, amount, due_date, payment_status, paid_date)
- **HOSTEL** (hostel_id, hostel_name, gender, total_capacity, warden_name, warden_phone)
- **HOSTEL_ROOM** (room_id, *hostel_id [FK]*, room_number, capacity, current_occupancy, floor_number)
- **HOSTEL_ALLOCATION** (allocation_id, *student_id [FK]*, *room_id [FK]*, allocation_date, vacation_date)
- **LIBRARY_BOOK** (book_id, isbn, title, author, *department_id [FK]*, total_copies, available_copies)
- **LIBRARY_TRANSACTION** (transaction_id, *book_id [FK]*, *student_id [FK]*, issue_date, due_date, return_date, fine_amount)
- **COMPANY** (company_id, company_name, industry_type, contact_email, tier)
- **PLACEMENT_DRIVE** (drive_id, *company_id [FK]*, drive_date, package_lpa, min_cgpa, venue)
- **PLACEMENT_REGISTRATION** (registration_id, *drive_id [FK]*, *student_id [FK]*, status)
- **APP_USER** (user_id, email, password_hash, plain_password, role, linked_id, full_name, created_at)

## 7. Database Tables, Attributes, Constraints & Sample Data

Below are detailed structural specifications for primary operational tables verified live in the Azure PostgreSQL instance (Total Student Count: 1,938; Courses: 330; Faculty: 73; Books: 200).

### Table 7.1: student (Entity Table)

| Attribute Name | Data Type | Constraint / Key | Sample Record Value |
|---|---|---|---|
| student_id | INTEGER | PRIMARY KEY (AUTO_INC) | 1 |
| roll_number | VARCHAR(30) | UNIQUE, NOT NULL | CB.SC.U4AIE24026 |
| first_name | VARCHAR(50) | NOT NULL | Sasank |
| last_name | VARCHAR(50) | NOT NULL | Reddy |
| email | VARCHAR(100) | UNIQUE, NOT NULL | cb.sc.u4aie24026@cb.students.amrita.edu |
| gender | VARCHAR(10) | CHECK (gender IN ('Male','Female','Other')) | Male |
| department_id | INTEGER | FOREIGN KEY (department) ON DELETE RESTRICT | 2 (AI & Data Engineering) |
| current_year | INTEGER | CHECK (current_year BETWEEN 1 AND 5) | 2 |
| cgpa | NUMERIC(4,2) | CHECK (cgpa BETWEEN 0.00 AND 10.00) | 8.75 |
| status | VARCHAR(20) | DEFAULT 'Active' | Active |

### Table 7.2: course (Academic Entity)

| Attribute Name | Data Type | Constraint / Key | Sample Record Value |
|---|---|---|---|
| course_id | INTEGER | PRIMARY KEY (AUTO_INC) | 364 |
| course_code | VARCHAR(20) | UNIQUE, NOT NULL | 22AIE303 |
| course_name | VARCHAR(150) | NOT NULL | Database Management Systems |
| credits | INTEGER | CHECK (credits BETWEEN 1 AND 10) | 4 |
| department_id | INTEGER | FOREIGN KEY (department) ON DELETE SET NULL | 2 |
| course_type | VARCHAR(30) | CHECK (course_type IN ('Core','Elective','Lab','Project')) | Core |

### Table 7.3: app_user (Security & Role-Based Access Control)

| Attribute Name | Data Type | Constraint / Key | Sample Record Value |
|---|---|---|---|
| user_id | INTEGER | PRIMARY KEY (AUTO_INC) | 101 |
| email | VARCHAR(100) | UNIQUE, NOT NULL | admin@campusos.edu |
| password_hash | VARCHAR(255) | NOT NULL (pbkdf2:sha256) | pbkdf2:sha256:10000$... |
| role | VARCHAR(20) | CHECK (role IN ('Admin','Faculty','Student')) | Admin |
| linked_id | INTEGER | REFERENCES student / faculty id | NULL (System Admin) |

# 8. Conclusion and Future Scope

**8.1 Conclusion:** The CampusOS Smart University Digital Twin successfully demonstrates an end-to-end cloud-native enterprise database system. Built on PostgreSQL 18 and Microsoft Azure, the platform bridges mathematical relational theory with real-world high-throughput university operations. Strict Boyce-Codd Normal Form (BCNF) normalization eliminated anomalies, while cascading foreign keys, check constraints, and role-based authorization protect institutional data integrity. The integration of containerization with Docker, live telemetry with Azure Application Insights, and an interactive 3D frontend delivers an intuitive, secure, and production-ready solution.

**8.2 Future Scope & Extensions:**

- **Automated Attendance via Edge Vision:** Integration with campus camera feeds to log attendance records directly into PostgreSQL using real-time edge facial recognition.
- **AI-Powered Predictive Academic Warning:** Implementing regression models to identify students with declining attendance and marks trends, automatically flagging them for faculty mentoring.
- **Kubernetes Microservices (AKS):** Transitioning from monolithic container deployment to microservices orchestrated on Azure Kubernetes Service (AKS) with automated horizontal pod autoscaling.
- **Read Replicas & Connection Pooling:** Configuring Azure PostgreSQL PgBouncer connection pooling and read replicas across geographical regions to support hundreds of thousands of concurrent portal users during semester result announcements.